

From Vibecoding to Deterministic Systems

safrano9999 and the 2026 Pattern for Model-Assisted Software

safrano9999

Rafael Felix Remy

Vienna (Wien), Austria

June 8, 2026

This paper studies a pragmatic development pattern for 2026, where “vibecoding” is treated as a useful starting condition rather than a finished engineering method. Strong coding models are used to accelerate the derivation of deterministic programs, while the resulting systems remain inspectable, service-oriented, and operationally boring. The safcontainer environment is used as a case study. It combines bare-metal Python services, container images, OpenClaw gateway plugins, FastAPI-based Web UIs, injected runtime configuration, GitHub tag-based releases, and channel-native webhook delivery. The central argument is that model-assisted coding becomes production-relevant only when the generated artifacts are pushed back into a disciplined architecture: explicit ports, explicit configuration, small plugin surfaces, reproducible images, and no hidden bridge logic.

1 Introduction

The working hypothesis of this paper is that vibecoding in 2026 is most useful when it is treated as a compiler for operational intent rather than as a runtime dependency. Strong models such as Codex-class coding agents can help derive programs, tests, configuration surfaces, and deployment scripts from rough requirements. The resulting software, however, should be deterministic where deterministic behavior is available. In this project, Python is the preferred implementation language for that transition: it is readable, fast enough for the targeted automation tasks, and has strong libraries for web services, mail handling, report generation, databases, and LLM client calls.

The safcontainer system¹ is the case study. It includes bare-metal programs, containerized deployments, OpenClaw gateway plugins, Telegram delivery, FastAPI Web UIs, GitHub release automation, Tailscale networking, and tmux-based operational supervision. The system is not built around a single monolith. Instead, it is a kit of small programs and shared scripts whose interfaces are constrained by environment variables, config examples, webhook paths, and generated release artifacts.

The resulting consensus is practical: new automation work should start from this pattern whenever possible. A repository begins with a bare-metal executable path, a minimal configuration example, a predictable environment loader, a small webhook surface, and an optional container wrapper. This saves time because later integration work is no longer invented repeatedly. The developer does not

¹Local development root: <https://github.com/safrano9999>.

have to rediscover how to inject secrets, expose a port, build an image, publish a plugin archive, or route a Telegram notification. Those decisions become defaults. Vibecoding remains useful, but it operates inside a structure that is already known to deploy.

This paper therefore treats vibecoding as a socio-technical loop: a human states intent, a model proposes and edits code, and the repository structure forces the result back into deterministic, reviewable, tagged, and deployable units.

2 Background

The project combines three operational traditions.

First, it keeps Unix-style separability. Each service can run directly on bare metal and has its own configuration, logs, and runtime dependencies. Second, it uses containers for reproducibility and deployment isolation. Containerfiles and Quadlet units provide stable process boundaries and systemd integration (Docker Documentation 2026; Podman Documentation 2026). Third, it uses OpenClaw as a gateway for chat channels, slash commands, webhooks, and scheduled execution (OpenClaw Project 2026). In the OpenClaw-specific containers, the gateway may run without an AI harness; deterministic plugins are still useful because they expose Telegram, webhook, and cron functionality.

Two shared conventions are central. The first is the Python header file, `python_header.py`, which normalizes path discovery, environment loading, and repository-local execution for bare-metal scripts. The second is `config.sh`, a shared configuration program that reads `env.example` and `config.conf_example`, asks for missing values, and writes runtime files. These two artifacts are not copied by hand across repositories. They are hardlinked from the shared SCRIPTS repository so that fixes are structural rather than duplicated.

3 Architecture

The architecture separates application logic, gateway integration, and deployment mechanics.

3.1 Container and Bare-Metal Co-Design

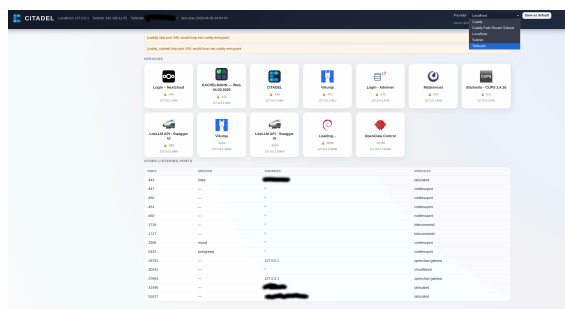
The same repositories are expected to work in two modes. Bare-metal mode is used for development, debugging, and direct service operation. Container mode stages the same repositories into images and injects configuration at runtime. This design avoids a common failure mode in automation systems: the container becomes the only executable form, while the source repository loses direct operability. In safcontainer, the bare-metal path remains first-class.

The shared Python header supports that model. It makes scripts independent of the caller's current directory, keeps repository roots predictable, and lets environment injection work the same way inside and outside containers. This is especially important for services such as ZEROINBOX, where credentials and provider definitions must be runtime state rather than baked image content.

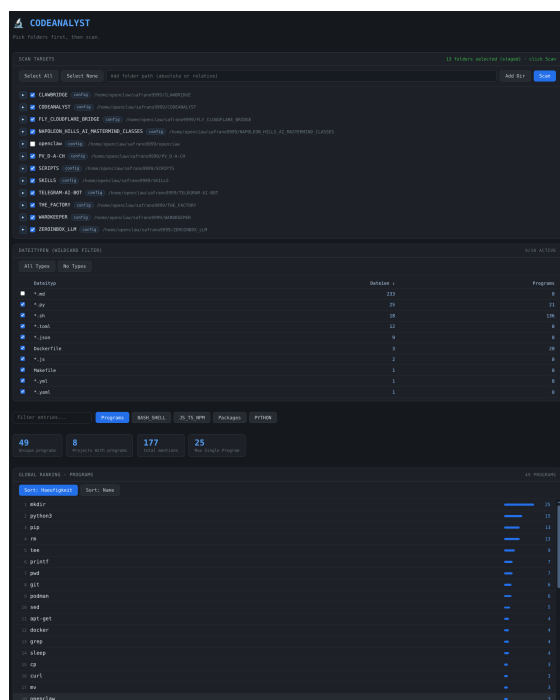
3.2 Service Inventory

3.3 FastAPI and Port Discipline

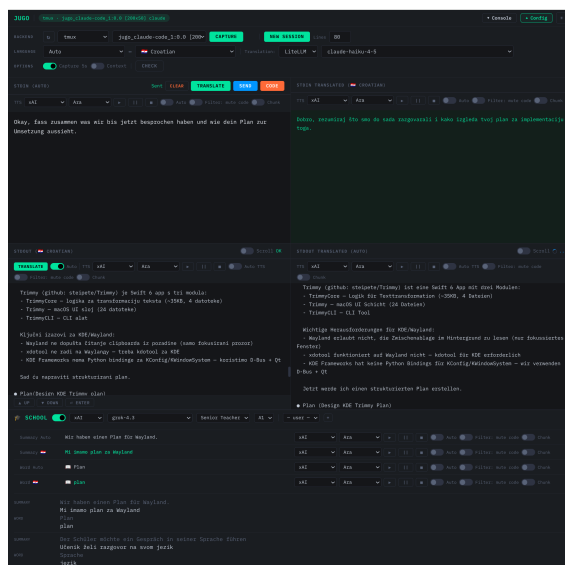
The Web UI side of the system is intentionally explicit. FastAPI is used for small administrative surfaces because it gives Python services a typed HTTP API, automatic OpenAPI metadata, and good ergonomics for forms and JSON handlers (FastAPI Project 2026). KACHELMANN is the clearest example: it exposes



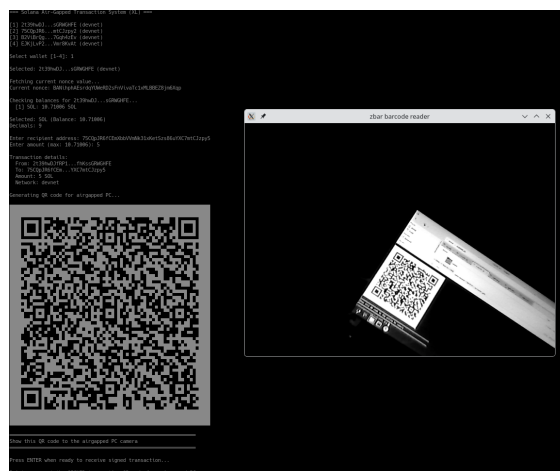
CITADEL



CODEANALYST



JUGO



SOLANA AIRGAPPED DEBIAN WORKFLOW

Figure 1: Screenshots and reference images from non-OpenClaw repositories that are staged into the Fedora development image.

Table 1: Main repositories staged into the Fedora/OpenClaw environment.

Repository	Role	Interface
DAILYNEWS	Daily PDF/news report generation	Webhook, slash command
CALENDAR	CalDAV agenda summary	Webhook, slash command
ZEROINBOX	Mail sorting and PDF reporting	Webhook, slash command
KACHELMANN	Routine tracking and dashboard	Webhook, slash command, Web UI
PV_D-A-CH	Photovoltaic/geospatial tooling	Standalone service
NAPOLEON HILLS AI	Structured knowledge workflows	Standalone service
MASTERMIND CLASSES		
NaturalGrounding Tiktok Ying Video Manager	Media workflow backend	Standalone service
SOLANA AIRGAPPED	Offline crypto workflow support	Standalone workflow
DEBIAN WORKFLOW		
CITADEL	Security/crypto operations	Standalone tooling
CODEANALYST	Code analysis support	Standalone tooling
JUGO	Domain-specific automation	Standalone tooling
VikAI	Vikunja/OpenClaw agent integration	OpenClaw-adjacent service
SCRIPTS	Shared build/config library	Hardlinked infrastructure

a Web UI on a configured port, while the OpenClaw gateway remains on its own port. The container config maps these ports explicitly, for example by separating a KACHELMANN Web UI port from the OpenClaw gateway port. This prevents the gateway from becoming a general-purpose application server.

Database access follows the same principle. SQLAlchemy is used where a service needs to target SQLite, PostgreSQL, MariaDB/MySQL, or other relational backends without changing service-level code (SQLAlchemy Project 2026). The database URL is runtime configuration; the image does not decide whether a deployment uses SQLite for a local test or PostgreSQL for the real host.

3.4 OpenClaw as Gateway, not Monolith

OpenClaw’s main advantage in this architecture is not that it can host an AI agent. Its main advantage is that it already solves gateway problems that would otherwise be rebuilt badly in each small service. It provides channel binding, slash commands, webhook endpoints, gateway authentication, cron-style execution, session routing, and a single place where delivery status can be observed (OpenClaw Project 2026). Telegram is only one concrete channel in the current deployment. The same pattern is meant to remain protocol-flexible: Telegram, Discord, webhooks, local control UIs, or future chat transports can be selected at the gateway boundary without rewriting the deterministic service. This turns service integration into a small plugin contract instead of a messaging rewrite.

The webhook boundary is particularly valuable. A plugin can expose `/plugins/dailynews`, `/plugins/zeroinbox/run`, or a similar endpoint. A container-generated full-run script can then call those endpoints with a gateway token. The script does not know Telegram, PDFs, mailboxes, or routine state. It only knows the OpenClaw gateway URL. This is the difference between orchestration and bridging. Orchestration calls stable interfaces; bridging duplicates delivery logic and becomes a second application.

This channel independence is the reason delivery configuration belongs in the plugin schema. A plu-

gin can deliver to the configured channel when the gateway knows the target; the deterministic service does not need to know which chat protocol was selected. The result is a replaceable communication layer around a stable program.

For development, the gateway also gives immediate feedback. A service can be tested from the shell with a single `curl` call, from Telegram through a slash command, from another supported chat protocol, from OpenClaw cron, or from a generated full-run script. These are different entrypoints into the same deterministic logic, which reduces debugging time and makes failures easier to classify.

3.5 Bias and Determinism

The architecture distinguishes between tasks where bias is acceptable and tasks where deterministic behavior is required. Daily research, trend discovery, and news aggregation can tolerate a controlled amount of bias because the outputs are interpretive and often built from external APIs. In those workflows, model assistance can be useful as long as sources and timestamps remain visible.

Mail handling, account configuration, routine state, and database migrations are different. These workflows should be deterministic first. When semantic classification is useful, the preferred pattern is a service-owned LiteLLM call, ideally against a local or explicitly configured model. The gateway should not silently turn deterministic infrastructure into a general chat-completion surface.

4 Implementation

4.1 Configuration as Source of Truth

The configuration model deliberately distinguishes secret and non-secret runtime state. Secret values live in `.env`. Non-secret operational settings such as ports, image names, crontab schedules, and optional volumes live in `config.conf`. Both files are generated from examples. The examples are merged from the staged repositories, so a container that includes more repositories asks for more configuration and a smaller container asks for less.

The merge step is intentionally mechanical. Each repository contributes `env.example`, `config.conf_example`, and `requirements.txt` where needed. The container setup collects these inputs, merges them, runs the shared `config.sh`, and then renders `compose.yml` and Quadlet container files. The resulting design has one runtime whitelist: if a variable is needed, it must appear in the relevant example. Ad-hoc writing of runtime secrets into plugin-local files is avoided.

The shared `config.sh` treats `container.example` as a third input for container-only runtime values and can skip that layer with `--no-container`. `#required:` marks the next variable as mandatory, while normal example values act as prompt defaults.

When the same key appears across layers, `config.sh` migrates an existing value from `.env` into the generated `config.conf`, and from `config.conf` or `.env` into `container.conf`, so a shared variable is entered once instead of duplicated across the three example files.

Required values may use `example: openssl rand -hex N` or `example: openssl rand -base64 N`; in that case the operator can either enter a value or let `config.sh` generate one without printing the secret. The special `DISPLAY` key opens a GUI environment flow that fills `DISPLAY`, `NO_AT_BRIDGE`, and `XDG_RUNTIME_DIR` by autodetection or manual entry.

`#blank-if: KEY=value A B` activates deterministic blank autofill for dependent variables, for example unused database fields when SQLite is chosen. Database backend values are normalized, and the first fresh multi-database configuration can apply one chosen backend to all repositories or fall back to per-repository prompts.

Provider selector variables ending in `_PROVIDER` are backed by `provider.conf` sections such as `[provider.gmail]`. Numeric prompt choices are normalized back to provider names, which keeps account examples small while preserving explicit runtime values.

`#mount-if: KEY=value PATH` and `#mount-bind: KEY` declare repo-relative bind mounts for generated Compose and Quadlet files. Repository SOT Markdown files listed in `.gitignore` with a `_SOT.md` suffix are mounted as files so local source-of-truth documents survive container runs.

`#publish-host: KEY` selects the host address used for generated port bindings. The container renderer also derives mappings from `_PUBLISH_PORT` plus the matching internal `_PORT`, and passes through `_VOLUMES`, `_DEVICES`, and `_CAPABILITIES`.

4.2 Hardlinked Shared Scripts

The `SCRIPTS` repository acts as a library-like repository. Files such as `config.sh`, `python_header.py`, merge helpers, Tailscale services, OpenClaw service snippets, and container helper functions are linked into dependent repositories with hardlinks. This reduces accidental drift between projects. A fix to the OpenClaw cron setup or the environment parser is made once, then re-linked across consumers.

4.3 Plugin Bundles and Full-Run Scripts

OpenClaw plugins are staged through a helper function, `safrano9999_OC_plugins`. The function receives repository names, downloads or stages the repositories, prepares Python virtual environments, installs the OpenClaw plugins, and optionally generates a full-run shell script. That generated script is intentionally simple: it contains the selected webhook `curl` calls and nothing else.

This rule is architectural rather than aesthetic. Telegram delivery belongs to the plugin and the OpenClaw channel layer. The full-run script must not become a message bridge. A correct run returns structured plugin payloads, often with `delivered: true`, and lets each plugin handle channel-native delivery.

4.4 Time Saved by Starting from the Pattern

The biggest practical benefit is that each new repository starts with known answers. The developer already has a place for runtime secrets, non-secret configuration, provider catalogs, initialization scripts, check scripts, release archives, and plugin manifests. A new service therefore begins with the domain problem rather than with scaffolding. This is where strong models provide the most leverage: they can generate the first working Python implementation inside a repository whose operational boundaries already exist.

The pattern also prevents common regressions. If a variable is needed at runtime, it must appear in the example files. If a plugin needs Telegram delivery, the manifest must expose a delivery configuration schema. If a container needs a new startup behavior, it must pass through the shared container helper or the explicit entrypoint. These constraints turn many integration mistakes into visible diffs.

4.5 Repository Skill Skeleton

The repository includes a Codex skill skeleton at `skills/safrano9999-project-skeleton/SKILL.md`. The skill is not a runtime dependency. It is a compact onboarding protocol for future coding agents. It explains how to start a new `safcontainer` project in either Web-UI-only mode or OpenClaw-plugin mode, how to hardlink the shared scripts, how to expose `env.example` and `config.conf_example`,

and how to keep generated projects compatible with the existing merge, config, tag, and release logic. The point is to make the desired architecture available at the first prompt, before ad-hoc scaffolding appears.

4.6 Tags, Release Zips, and Images

GitHub Actions are used as the release boundary (GitHub Docs 2026b). Plugin repositories produce `*-latest.zip` archives from tags, while the container repository builds images from the selected staged artifacts. GitHub releases are tag-based (GitHub Docs 2026a); the operational convention is to force-update latest and create a date tag such as `2026.6.8`. This gives humans a stable moving pointer and a dated checkpoint.

This tagging step is standardized through a small shared helper, `tag.sh`. In the development workspace the file is hardlinked from `SCRIPTS` into each plugin repository and into the container subdirectories, so the operator can run `./tag.sh` from the repository being worked on. The script derives the date tag from the local timestamp, deletes the local and remote copy of that tag, recreates it on the current HEAD, and pushes it. A `--check` mode does not mutate tags; it prints and runs the matching `gh run list --branch <date-tag>` command. The hardlink is a workspace convention for keeping the helper identical; Git still stores the file content normally, so cloned repositories remain usable.

The image build layer then pulls the plugin archives, stages them into the container context, pre-builds Python environments, and installs OpenClaw plugin metadata. When a plugin archive is stale, the symptom is visible at the gateway boundary: the plugin still runs, but the container may miss updated delivery schema. This is why artifact inspection is treated as part of the build loop.

SQLite persistence. Each repository declares its backend key in its local `env.example`; when that resolved value is `sqlite`, the shared Python header creates `./sqlite`. Container generation mounts that directory through a repository-specific named volume, while bare-metal execution uses the same relative path directly.

Named-volume prelinking. Optional state uses *prelinking*: when a named-volume mount exists, initialization replaces the selected runtime paths with symbolic links into that volume without pre-creating files or directories; without the mount, those paths remain ephemeral. The operation is *idempotent*, so every container start can safely recreate the same links.

Repeatable configuration groups. A repository marks repeatable fields with `#repeat-group`; the shared `config.sh` then offers `skip/new` and writes the next indexed CALENDAR, ZEROINBOX, or OPENAI_V1 group without container-specific prompting logic.

Container-scoped configuration names. An exact `#CONTAINER-NAME` directive immediately above `CONTAINER_NAME` makes the shared `config.sh` request the instance name before every other value. The selected name scopes `NAME.env`, `NAME_config.conf`, `NAME_container.conf`, `NAME_build.conf`, `NAME-compose.yml`, and `NAME.container`; Compose, Quadlet, SQLite persistence, optional named volumes, and the configuration audit therefore resolve one coherent instance without hardcoded container names.

Quadlet completion helper. The hardlinked `quadlet_finish.py` helper in the `SCRIPTS` SOT prints the exact commands for Compose startup, linking the generated Quadlet into the user's `systemd` directory, running `systemctl --user daemon-reload`, and restarting the selected service. It deliberately reports these host actions instead of executing them implicitly.

Provider authentication as SOT. Injected `*_API_KEY` variables are the single source of truth for model-provider authentication in both OpenClaw and Hermes: Hermes performs its native provider discovery directly, while OpenClaw maps each key through its own model-status output and registers every available model deterministically. Sakana is handled as an explicit provider because OpenClaw reports its authentication through `models.json`; Codex remains the file-based exception and is enabled only when the prelinked `auth.json` exists.

Browser-native Markdown workspace. Flatnotes provides a FastAPI-backed browser editor for the Markdown files in the persistent Hermes Obsidian directory. Its internal and published ports follow the same generated configuration and service-discovery rules as the other Web UIs, so CITADEL can expose the listener through localhost, Tailscale, or Cloudflare without changing the application.

Generated welcome reference. Container setup derives a value-free `ref.conf` from the merged examples and excludes every variable marked `#secret`. The `WELCOME` plugin renders this reference with the architecture paper, serves it through `/start`, `/welcome`, and the existing OpenClaw gateway webhook, and runs first in the deterministic full run without adding another listener.

5 Evaluation

Evaluation is operational rather than benchmark-driven. The relevant question is not whether a model can answer a prompt; it is whether the derived software can be restarted, scheduled, inspected, and trusted.

5.1 Gateway Execution

The primary check is the full-run sequence. It executes the four gateway-facing services through their webhook endpoints:

1. `DAILYNEWS` generates a PDF report and returns a media path.
2. `CALENDAR` reads CalDAV configuration and returns an agenda summary.
3. `ZEROINBOX` sorts unread mail and returns a mailbox-level status.
4. `KACHELMANN` reports missing routine tasks and exposes a Web UI.

A successful channel-integrated run returns structured payloads and `delivered: true` where the plugin has a configured Telegram delivery target. The absence of this field is a configuration signal, not a reason to introduce an external bridge.

The evaluation therefore includes a negative rule: if only some plugins deliver to Telegram, the first diagnostic step is schema and config inspection, not the addition of a wrapper. In the observed system, stale plugin release archives could run successfully but miss their newer delivery schema. Rebuilding the release archive and image fixed the delivery path without changing the full-run script. This is a useful example of the architecture paying for itself: the error was localized to artifact freshness.

5.2 Operational Substrate

The Fedora image is the broad development and operations container. It includes tmux for supervised sessions (tmux Project 2026), GitHub CLI for repository work, Tailscale for tailnet access (Tailscale Docs 2026), OpenClaw, optional Codex harness support, and the staged safcontainer repositories. A smaller Debian OpenClaw image focuses on the gateway plugin use case. Both images share the same configuration and script logic where behavior is genuinely common.

This distinction matters. Fedora is allowed to be a heavier workstation-like container. The OpenClaw gateway container should remain focused: install the plugins, start the gateway, run optional Tailscale, and expose the configured ports.

6 Security Considerations

6.1 No Hidden Runtime Harness

The OpenClaw-only container intentionally supports deterministic gateway usage without configuring an AI model provider. Some services may call LiteLLM directly through their own Python code where completion calls are part of the service logic (LiteLLM Project 2026). That is different from giving the OpenClaw gateway a model harness by default. The separation reduces the blast radius of prompt injection and keeps deterministic command paths deterministic.

6.2 Runtime Secrets and Optional State

Secrets are injected at runtime, not baked into images. This makes containers ephemeral by default: a rebuilt container should be able to receive the same configuration again from `.env`, `config.conf`, mounted provider files, or host-managed secrets. Optional volumes for Codex CLI authentication, OpenClaw model authentication, Tailscale identity, logs, and reports are documented as commented configuration. They can be enabled deliberately, but the default image does not require persistent model credentials. The optional volume model is therefore facultative rather than hidden state.

6.3 Crypto Workflows

The crypto-related repositories are treated with extra conservatism. The `SOLANA_AIRGAPPED_DEBIAN_WORKFLOW` project is modeled as an offline workflow, where key material and signing steps are separated from networked automation. Solana key material and signing concepts are documented in the Solana ecosystem (Solana Foundation 2026; Solana Documentation 2026). In the safcontainer model, crypto tooling belongs to explicit, inspectable workflows rather than invisible background automation.

7 Conclusion

Vibecoding is productive when it produces systems that can survive outside the chat session that created them. The safcontainer/OpenClaw architecture shows one way to enforce that outcome. Strong models help derive code, but the system accepts only deterministic artifacts: Python services, FastAPI endpoints, explicit ports, merged configuration examples, hardlinked shared scripts, release archives, tagged images, and webhook payloads.

The key lesson is that the useful boundary is not between AI and non-AI software. The useful boundary is between derivation and operation. Models can participate in derivation. Operation should remain explicit, observable, and boring.

8 n8n as the Next Integration Layer

n8n is the planned workflow layer for cases where a visual automation graph is more useful than another Python service (n8n Documentation 2026). It should remain an orchestrator: call documented webhooks, receive structured payloads, and keep credentials in its credential system.

Bias is acceptable in daily research workflows when n8n calls APIs, collects signals, and produces a human-readable briefing. For email, account state, or routine state, the safer pattern remains deterministic service logic, with optional LiteLLM calls owned by the service itself. Local models are especially attractive for mail-oriented workflows because the data boundary is clearer and the gateway does not need to become the model harness.

A concrete future service is a 24/7 streaming news and research channel. In that design, n8n would periodically fetch fresh signals from news APIs, social feeds, RSS sources, market data, and project-specific endpoints. A Python service would normalize and score the incoming material, while a rendering pipeline would produce a continuously refreshed TV-like stream: headlines, charts, short summaries, source links, and scheduled deep-dive segments. Bias is acceptable here because the product is editorial and exploratory, but the pipeline should still preserve source URLs, timestamps, and provenance. The same gateway pattern can then publish snapshots to chat, trigger manual refreshes through slash commands, or expose webhooks for external screens.

References

- Docker Documentation (2026). *Dockerfile Reference*. URL: <https://docs.docker.com/reference/builder> (visited on 06/08/2026).
- FastAPI Project (2026). *FastAPI Features*. URL: <https://fastapi.tiangolo.com/features/> (visited on 06/08/2026).
- GitHub Docs (2026a). *About Releases*. URL: <https://docs.github.com/articles/about-releases> (visited on 06/08/2026).
- (2026b). *Workflow Syntax for GitHub Actions*. URL: <https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax> (visited on 06/08/2026).
- LiteLLM Project (2026). *LiteLLM Documentation*. URL: <https://docs.litellm.ai/> (visited on 06/08/2026).
- n8n Documentation (2026). *n8n Workflows Documentation*. URL: <https://docs.n8n.io/workflows/> (visited on 06/08/2026).
- OpenClaw Project (2026). *OpenClaw CLI Cron Documentation*. URL: <https://docs.openclaw.ai/cli/cron> (visited on 06/08/2026).
- Podman Documentation (2026). *podman-quadlet*. URL: <https://docs.podman.io/en/latest/markdown/podman-quadlet.1.html> (visited on 06/08/2026).
- Solana Documentation (2026). *Program Derived Addresses*. URL: <https://solana.com/docs/core/pda> (visited on 06/08/2026).
- Solana Foundation (2026). *Keypair*. URL: <https://solana-foundation.github.io/solana-web3.js/v1.x/classes/Keypair.html> (visited on 06/08/2026).
- SQLAlchemy Project (2026). *SQLAlchemy Features*. URL: <https://www.sqlalchemy.org/features.html> (visited on 06/08/2026).

- Tailscale Docs (2026). *Install Tailscale on Linux*. URL: <https://tailscale.com/docs/install/linux> (visited on 06/08/2026).
- tmux Project (2026). *tmux Source Repository*. URL: <https://github.com/tmux/tmux> (visited on 06/08/2026).